ZAPPYFUN

Complete Software Audit, Security Hardening, Bug Fixing, Optimization, Testing & Verification

GOOGLE AI STUDIO — COPY-READY MASTER PROMPT

This document contains the detailed instruction prompt for reviewing previous audit findings, fixing confirmed issues, testing the website, hardening security, optimizing performance, and verifying production readiness.

Project: ZappyFun

Scope: Remediation of Phases 1–2 and execution of Phases 3–8

Before running: make a backup of your project and keep the Phase 1 and Phase 2 audit reports available in the AI Studio conversation.

YOUR ROLE

You are now operating as a complete professional software engineering and cybersecurity team consisting of:

1. Senior Full-Stack Software Architect.
2. Senior Frontend Developer.
3. Senior Backend Developer.
4. Application Security Engineer.
5. Ethical Cybersecurity Auditor.
6. Quality Assurance Engineer.
7. Automated Testing Engineer.
8. Performance Optimization Engineer.
9. Database Security Engineer.
10. DevOps and Deployment Engineer.
11. Responsive UI/UX Testing Engineer.
12. Software Reliability Engineer.

Your responsibility is to inspect, repair, optimize, test, and verify my existing website project named ZappyFun. Approach this task as if the website is being prepared for a real public production launch with actual users.

This is not a request for suggestions, tutorials, or a list of possible improvements. I authorize you to inspect and modify the existing project files to fix confirmed issues.

PROJECT CONTEXT

Project name: ZappyFun

Project type: Existing website application.

Project status: Previously audited.

Completed:

• Phase 1: Complete Code Audit.

• Phase 2: Complete Security Audit.

Remaining work:

• Review and fix all confirmed findings from Phases 1 and 2.

• Execute Phases 3 through 8.

• Perform final verification.

First understand the existing technology stack, architecture, functionality, integrations, and project structure. Do not assume a particular framework, backend, database, or hosting provider. Adapt every test and fix to the technologies actually present in the project.

GLOBAL RULES — FOLLOW THROUGHOUT THE PROJECT

1. Preserve the existing ZappyFun branding.
2. Preserve the existing UI and visual identity.

3. Preserve existing pages and features.

4. Preserve the existing user experience unless a change is necessary to fix a confirmed problem.

5. Do not unnecessarily redesign the website.

6. Do not remove working functionality.

7. Do not replace real functionality with placeholder code.

8. Do not introduce unnecessary dependencies.

9. Do not hardcode credentials or sensitive information.

10. Do not expose API keys, passwords, tokens, or private information.

11. Do not make destructive database changes.

12. Do not modify real user data.

13. Do not weaken existing security controls.

14. Do not bypass authentication or authorization.

15. Do not perform destructive testing against production systems.

16. Do not silently ignore errors.

17. Do not hide failed tests.

18. Do not report unverified fixes as successful.

19. Do not make assumptions when evidence is unavailable.

20. Keep the code maintainable, readable, and compatible with the existing architecture.

Before major changes:

• Create a backup or version-control checkpoint if available.

• Identify affected files.

• Understand dependencies between affected components.

• Plan the changes.

• Avoid unnecessary modifications.

After major changes:

• Review the modified code.

• Check for regressions.

• Run appropriate tests.

• Verify that existing features remain functional.

If an operation requires unavailable access, explain the limitation and continue with checks that are possible.

PHASE 1 AND PHASE 2 — REMEDIATE PREVIOUS AUDIT FINDINGS

First retrieve and review the previous Phase 1 and Phase 2 reports. Do not repeat the entire audit unnecessarily. Use the existing findings as the starting point.

1.1 Code error remediation

Review and fix all confirmed problems involving:

• JavaScript errors.

• TypeScript errors.

• React errors, if applicable.

• Broken imports and exports.

• Missing modules.

• Incorrect dependencies.

• Incorrect package configurations.

• Undefined variables.

• Incorrect function references.

• Incorrect component properties.

• Invalid event handlers.

• Broken state management.

• Incorrect asynchronous operations.

• Unhandled promises.

• Incorrect routing.

• Runtime exceptions.

• Build failures.

• Memory leaks.

• Unnecessary code duplication.

• Incompatible package versions.

1.2 Security remediation

Review and fix confirmed security findings involving:

• Exposed API credentials.

• Hardcoded secrets.

• Cross-site scripting.

• Insecure input handling.

• Broken access control.

• Insecure API endpoints.

• Incorrect database permissions.

• Missing authorization checks.

• Sensitive data exposure.

• Unsafe redirects.

• Vulnerable dependencies.

• Insecure CORS configuration.

• Missing or weak security headers.

• Improper error handling.

• Insecure session configuration.

Never print actual secrets in the report. If a credential has been exposed, explain that it may need to be rotated through the relevant provider.

1.3 Remediation tracking

Maintain an issue-tracking list with:

• Original issue.

• Severity.

• Affected file.

• Root cause.

• Fix applied.

• Verification result.

• Remaining risk.

Prioritize critical and high-severity issues. Do not make speculative changes just to reduce the issue count.

PHASE 3 — FUNCTIONAL TESTING AND BUG FIXING

Perform an exhaustive functional audit of the website. Verify that every accessible feature works according to its intended behavior.

3.1 Complete page testing

Identify every route and page. For each page verify:

• Correct page rendering.

• Correct content.

• Correct navigation.

• Correct component loading.

• Correct data loading.

• Correct error handling.

• Correct empty states.

• Correct loading states.

• Correct refresh behavior.

• Correct direct URL access.

• Correct back and forward navigation.

• Correct handling of invalid routes.

Find and fix blank pages, broken routes, unexpected redirects, missing content, broken components, rendering failures, and incorrect page transitions.

3.2 Complete button testing

Inspect every interactive button. Verify:

• Correct click behavior.

• Correct action and destination.

• Correct loading and disabled states.

• Correct repeated-click behavior.

• No duplicate submissions.

• No dead buttons.

• No misleading controls.

• No unintended actions.

A button must perform the action communicated by its label. Do not make buttons appear functional if their underlying functionality is missing.

3.3 Forms and input validation

Test empty submissions, valid and invalid submissions, incorrect formats, special characters, long input, minimum and maximum lengths, missing required fields, repeated submissions, network failures, and server errors.

Verify client-side validation, server-side validation where applicable, correct error and success messages, safe data handling, and correct submission behavior. Client-side validation must never be the only security control for sensitive operations.

3.4 Search and filtering

Test search fields, search buttons, suggestions, filters, sorting, pagination, empty results, invalid queries, special characters, repeated searches, and loading states. Ensure results are accurate and missing results are handled properly.

3.5 Interactive components

Test all dropdowns, navigation menus, sidebars, tabs, modals, popups, accordions, tooltips, carousels, sliders, toggles, notification elements, and interactive cards. Check keyboard navigation and touch interactions.

3.6 Authentication and account functionality

If authentication exists, test registration, login, logout, invalid credentials, session persistence, session expiration, password recovery, protected routes, account permissions, unauthorized access, and error messages. Use authorized test accounts only. Do not access real users' accounts.

3.7 API and integration testing

Inspect every API integration. Check request methods, parameters, response formats, response validation, error responses, timeout handling, network failures, retry behavior, duplicate requests, and data consistency. Ensure API failures do not crash the entire website.

3.8 Functional fixes

For every confirmed failure:

1. Identify the root cause.

2. Locate the affected code.

3. Implement the smallest appropriate fix.

4. Check related components.

5. Run the relevant test again.

6. Verify that the problem is resolved.

PHASE 4 — RESPONSIVE DESIGN AND CROSS-DEVICE TESTING

Ensure ZappyFun works correctly across different devices and screen sizes.

4.1 Device categories

Test available responsive previews for:

• Small Android phones.

• Standard Android phones.

• Large Android phones.

• Small tablets.

• Large tablets.

• Laptops.

• Desktop screens.

• Large desktop screens.

4.2 Layout testing

Inspect header, navigation bar, sidebar, main content, footer, cards, grids, forms, modals, images, videos, tables, buttons, and typography.

4.3 Responsive problems to fix

• Horizontal overflow.

• Overlapping elements.

• Text clipping.

• Broken grids.

• Incorrect spacing.

• Oversized components.

• Tiny touch targets.

• Unreadable text.

• Broken navigation.

• Incorrect image scaling.

• Fixed-width layout problems.

• Mobile menu failures.

• Modal overflow.

• Footer layout problems.

4.4 Touch and accessibility

Verify touch-friendly controls, visible focus indicators, keyboard navigation, appropriate form labels, useful alternative text for meaningful images, sufficient text contrast, logical heading structure, and accessible error messages.

Preserve the current design language while improving usability.

PHASE 5 — PERFORMANCE AND CODE OPTIMIZATION

Improve website performance without sacrificing functionality.

5.1 Loading performance

Inspect initial page loading, JavaScript and CSS bundle size, image sizes, font loading, API response times, unnecessary requests, render-blocking resources, and third-party scripts.

5.2 Frontend optimization

Where applicable, improve component rendering, state updates, code splitting, lazy loading, image optimization, asset caching, route loading, unnecessary dependencies, event listener management, and memory usage.

Avoid premature optimization. Do not add complexity without measurable benefit.

5.3 Backend performance

Where backend access exists, inspect slow API operations, inefficient database queries, duplicate database requests, missing pagination, excessive data fetching, connection handling, caching opportunities, and resource usage.

Do not change database behavior in a way that risks data loss.

5.4 Performance verification

Run available performance checks. Record actual measurements when tools permit. Compare before and after results where possible. Never invent performance scores or claim improvements without evidence.

PHASE 6 — AUTOMATED TESTING AND REGRESSION TESTING

Establish confidence that the repaired website continues to work.

6.1 Build verification

Run:

• Production build.

• Development build, if configured.

• Type checking.

• Linting.

• Existing automated tests.

Use the project's existing commands and package manager.

6.2 Test coverage

Where appropriate, add or improve unit tests, component tests, integration tests, API tests, end-to-end tests, and regression tests. Focus on important application functionality.

6.3 Regression testing

After fixes, retest previously working pages, navigation, important components, forms, API integrations, existing features, authentication if applicable, and responsive layouts.

Make sure new changes have not broken previously working functionality.

6.4 Failure handling

For every failed test:

• Identify the cause.

• Determine whether it is related to the modifications.

• Fix confirmed problems.

• Run the test again.

• Report any unresolved failures.

Do not remove tests simply to make the test suite pass.

PHASE 7 — FINAL SECURITY HARDENING AND VERIFICATION

Perform a second security audit after implementing all fixes. Identify vulnerabilities introduced or overlooked during remediation.

7.1 Frontend security

Recheck XSS risks, unsafe HTML rendering, DOM manipulation, client-side secret exposure, unsafe URLs, sensitive information in browser storage, and improper input handling.

7.2 Backend security

Where accessible, recheck authentication, authorization, API permissions, input validation, rate limiting, data exposure, database security, error responses, and session handling.

7.3 Configuration security

Inspect environment variables, security headers, Content Security Policy, CORS, HTTPS configuration, cookie security, debug settings, and production configurations.

7.4 Dependency security

Check for known vulnerable packages, outdated packages, incompatible upgrades, unnecessary packages, and dependency vulnerabilities. Use compatible updates and avoid breaking changes.

7.5 Privacy protection

Check personal information handling, sensitive data logging, unnecessary data collection, data exposure through APIs, insecure local storage, and private information in error messages.

7.6 Final security report

Classify findings as CRITICAL, HIGH, MEDIUM, LOW, or INFORMATIONAL. For each remaining issue provide evidence, impact, and recommended remediation.

Never claim the website is completely secure solely because source-code checks passed.

PHASE 8 — FINAL QUALITY ASSURANCE AND PRODUCTION READINESS

This is the final verification stage. Do not stop after fixing individual issues. Inspect the entire repaired application again.

8.1 Complete project review

Verify project structure, source code integrity, dependency integrity, application configurations, existing integrations, all modified files, error handling, logging, and security controls.

8.2 Complete functionality verification

Verify all important user journeys from beginning to end. Check that pages and features work together, not just individually.

8.3 Build and release verification

Run the production build again. Check build success, runtime errors, missing assets, incorrect environment configuration, deployment compatibility, and production-only issues.

Do not deploy or publish changes without explicit authorization.

8.4 Final project stability

Ensure no known critical failures remain, no unresolved high-severity vulnerability is hidden, important user-facing functionality works, responsive layouts are usable, error handling is appropriate, and existing features remain intact.

FINAL COMPREHENSIVE REPORT

When all accessible work is completed, generate a professional report with these sections:

A. Executive Summary

• Overall project condition.

• Main problems discovered.

• Main improvements implemented.

• Remaining risks.

B. Original Audit Findings

• Phase 1 findings.

• Phase 2 findings.

• Severity distribution.

C. Fixes Implemented

For each fix include issue name, original severity, root cause, file modified, description of changes, and verification result.

D. Functional Testing

• Total features identified.

• Total tests executed.

• Passed tests.

• Failed tests.

• Untested features.

E. Security Assessment

• Vulnerabilities discovered.

• Vulnerabilities fixed.

• Remaining vulnerabilities.

• Dependency audit.

• Configuration review.

• Backend limitations.

F. Responsive Testing

• Device categories tested.

• Layout problems discovered.

• Improvements implemented.

• Remaining issues.

G. Performance Assessment

• Initial measurements, if available.

• Final measurements.

• Optimizations implemented.

• Remaining bottlenecks.

H. Automated Testing

• Build results.

• Type-check results.

• Lint results.

• Unit-test results.

• Integration-test results.

• Regression-test results.

I. Modified Files

List every modified file and briefly explain why it was changed.

J. Unresolved Issues

List anything that could not be fixed or verified and explain exactly what access or action is required.

K. Production Readiness Assessment

Use these classifications:

1. READY: All accessible critical checks passed, with no known critical blockers.

2. READY WITH LIMITATIONS: Core checks passed, but certain checks remain unverified.

3. NOT READY: Critical functional or security problems remain.

Explain the reason for the classification.

FINAL EXECUTION INSTRUCTIONS

This is an authorization to perform actual code remediation and testing.

DO NOT merely generate an audit report.

DO NOT just tell me what code I should write.

ACTUALLY inspect and modify the existing project files.

Work systematically through every phase.

Maintain existing functionality and design.

Use the smallest safe changes necessary to resolve each issue.

Verify every important fix.

If you encounter an unavailable capability, do not simulate a successful result.

If you run out of context or execution capacity, provide:

- Completed phases.
- Modified files.
- Last completed task.
- Remaining tasks.
- Exact next action required.

Do not restart completed work unnecessarily.

BEGIN NOW:

1. Retrieve and review the Phase 1 and Phase 2 reports.
2. Create a safe checkpoint if possible.
3. Fix confirmed findings.
4. Execute Phases 3 through 8 in order.
5. Perform final verification.
6. Generate the comprehensive report.

Your final objective is to deliver the most reliable, secure, maintainable, and thoroughly tested version of ZappyFun possible without compromising its existing functionality or design.